PROJECT 3 · JUDGE THE JUDGE

AI judges wave through replies that promise too much

Customer: “Can I keep the car a few days longer?”

> A: You can usually extend your rental in the app or by phone, **as long as the car is not reserved for another customer.**

> B: You can usually extend your rental in the app or by phone. ~~[condition dropped]~~

The starting judge passes both. B commits SIXT to something the policy does not guarantee, and the judge catches only **7%** of dropped conditions on dev. A release gate built on it lets these replies ship.

INSIGHT

A judge that reads one reply cannot see what is missing.

So we make the absence explicit.

For every claim, call 1 lists the conditions and hedges the **policy** attaches to it and marks each one *kept*, *dropped* or *not applicable*. A dropped condition becomes a visible field instead of a gap.

0.067 → 0.667

dropped-condition catch rate, baseline → ours (dev, grouped 3-fold CV, shipped judge)

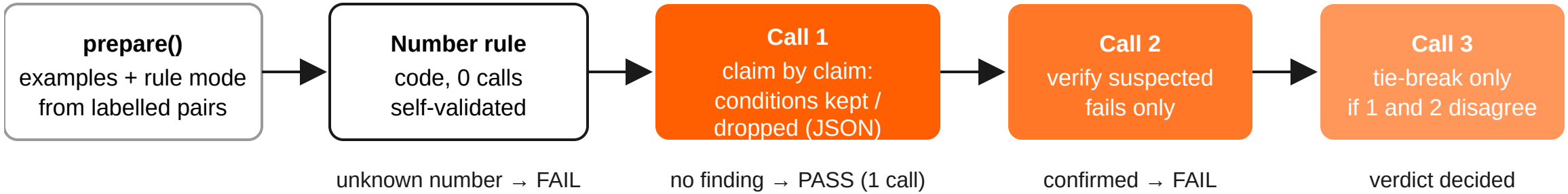
The rest is generic: certainty on a 0–3 scale (“usually” vs “always”), scope (some vs all branches), and numbers checked in code. Nothing about the criterion is hard-coded; examples and rule mode come from the labelled pairs.

Numbers from score.py reports: starter_pack/baseline_report.txt, results/*_report.txt

METHOD

Code for facts, the LLM for meaning, code for the verdict

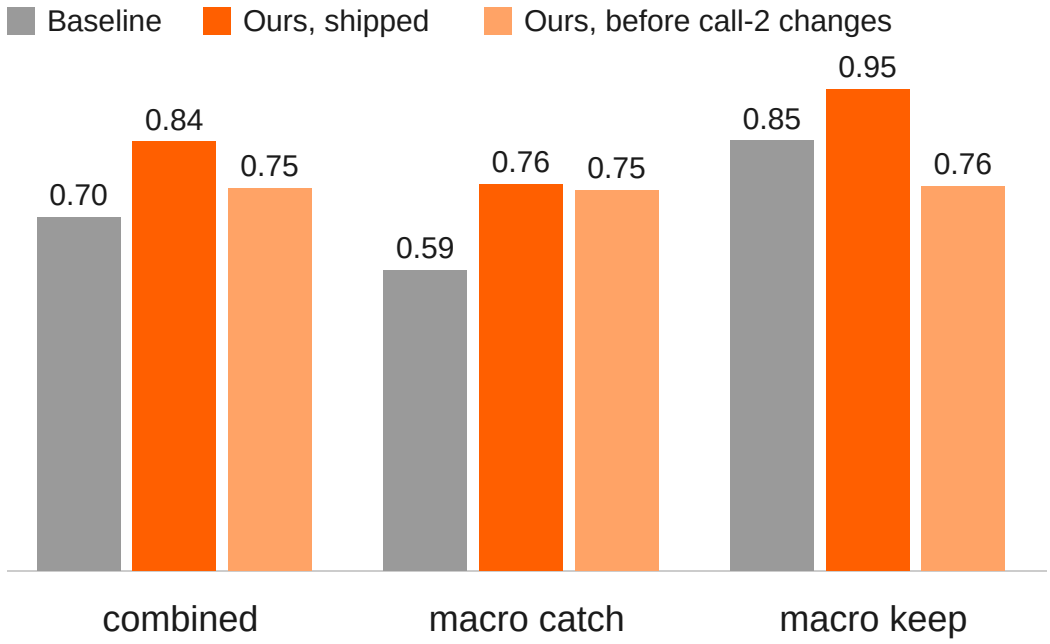
verdict computed in code · one reply per call · no state · ≤ 3 calls · claude-haiku-4.5, temperature 0



- **Self-validating rule:** a number not in the policy fails the reply, only if `prepare()` sees it never fire on a pass example of the current criterion; otherwise hint or off.
- **Verify only suspected fails** in a fresh context. Call 2 may confirm a fail only by quoting the reply's exact words (checked in code) and naming a concrete misleading case. If it proves none, the reply passes; a third call only if call 2's answer is unreadable. At most 2 calls in practice.
- Same code for the surprise criterion: only `--criterion` and `--dev` change.

RESULTS · DEV, GROUPED CROSS-VALIDATION, claude-haiku-4.5

More bad replies caught, and more good replies kept



crit.	judge	combined	catch	keep	kappa	repeat
1	Baseline	0.697	0.592	0.847	0.531	n/a
1	Ours, shipped	0.845	0.762	0.949	0.789	1.000
1	Ours, before call-2 changes	0.754	0.750	0.758	0.623	0.974
2	Baseline	0.652				
2	Ours, unchanged	0.883	0.867	0.900	0.788	1.000

What fixed keep: call 2 must prove a violation with the reply's own words, and an unproven finding passes (correct original passed 0.805 → 0.965, baseline 0.858). **Disclosed:** the last change (an unproven finding passes, no call 3) was made after the surprise criterion arrived and measured on both; it costs some criterion-1 catch on dropped conditions. **Caveat:** both call-2 changes were designed on dev errors, so dev gains are an upper bound. “repeat” = same verdict on two runs.

Sources: starter_pack/baseline_report.txt, results/dev_pairs_report.txt (shipped), results/exp_base_dev_dev_pairs_report.txt, results/c2_dev_pairs_report.txt

FOR SIXT

A release gate for chatbot changes, with the reason attached

- Run every candidate chatbot version against the policy before release; a reply that drops a condition or raises certainty is blocked.
- Each fail carries the quoted reply words and the policy clause, so a reviewer decides in seconds.
- The recipe takes any new criterion plus a few labelled examples, with no code change (shown live on the surprise criterion).

Next steps

- Close the gap on wrong numbers and wrong policy details, where the baseline still catches slightly more.
- Retrieve the closest labelled examples per reply instead of a fixed set.
- Measure transfer on edit types not in dev (leave-one-edit-type-out).